

```
class Banner
```

```
attr_accessible :horiz, :link, :visible, :image, :position  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
class Banner < ActiveRecord::Base
```

```
attr_accessible :horiz, :link, :visible, :image  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
protected
```

```
def assign_position
```

```
max = Banner.maximum(:position)
```

```
self.position = max ? max + 1 : 0
```

FULL STACK DEVELOPMENT

REACT – O PRINCÍPIO

AULA 02

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

O QUE VEM POR AÍ?

Nessa aula, aprenderemos conceitos de desenvolvimento básico de ReactJs. Você verá como inicializar um projeto ReactJS utilizando Vite, construção dos primeiros componentes, conceitos e aplicação de State e Props.

EMAND

HANDS ON

Nessa aula, os(as) estudantes aprenderão a inicializar um projeto ReactJS utilizando Vite. Também entenderão conceitos e aplicação de **State**, que são os dados mantidos internamente por um componente, os quais podem ser modificados ao longo do tempo, desencadeando atualizações na interface do usuário para refletir determinadas mudanças. Além disso, serão apresentados conceitos e aplicações sobre **Props**, que são meios de passar dados de um componente pai para um componente filho. Logo, as Props são utilizadas para transmitir informações e configurações entre componentes, contribuindo para a modularidade e reusabilidade do código.

SAIBA MAIS

Primeiramente, é importante entendermos a necessidade da criação de componentes. Componentes são criados para organizar e modularizar a interface do usuário de uma aplicação, encapsulam lógica, estado e apresentação facilitando o desenvolvimento, manutenção e reutilização do código. Eles contribuem para uma arquitetura mais compreensível, escalável e eficiente.

A necessidade da criação se faz quando algo se repete muito no projeto, utilizando a mesma funcionalidade e comportamento, então provavelmente aquele elemento deve virar um componente. Temos também a situação inversa, elementos que possuem funcionalidades específicas, por exemplo, um botão de upload que carrega uma lista de usuários em massa no banco de dados, tendo um comportamento único e, desta forma, se faz necessária a configuração do elemento "button" para um componente a parte.

Agora, vamos iniciar um projeto e demonstrar a funcionalidade das props e state?

Em seu terminal, vá para o diretório desejado e digite o comando:

```
npm create vite@latest
```

Seguindo os passos, informe o nome do Projeto [1], selecione o React como framework [2] e, por último, a linguagem base [3].

```
→ npm create vite@latest
Need to install the following packages:
create-vite@5.1.0
Ok to proceed? (y) y
[1] Project name: ... StatePropReact
[Pressione enter] Package name: ... statepropreact
[2] Select a framework: > React
[3] Select a variant: > - Use arrow-keys. Return to submit.
> TypeScript
  TypeScript + SWC
  JavaScript
  JavaScript + SWC
```

Figura 1 – Código 1
Fonte: elaborada pela autora (2024)

Ao acessar o repositório no VS Code, temos a seguinte aparência da estrutura do projeto:

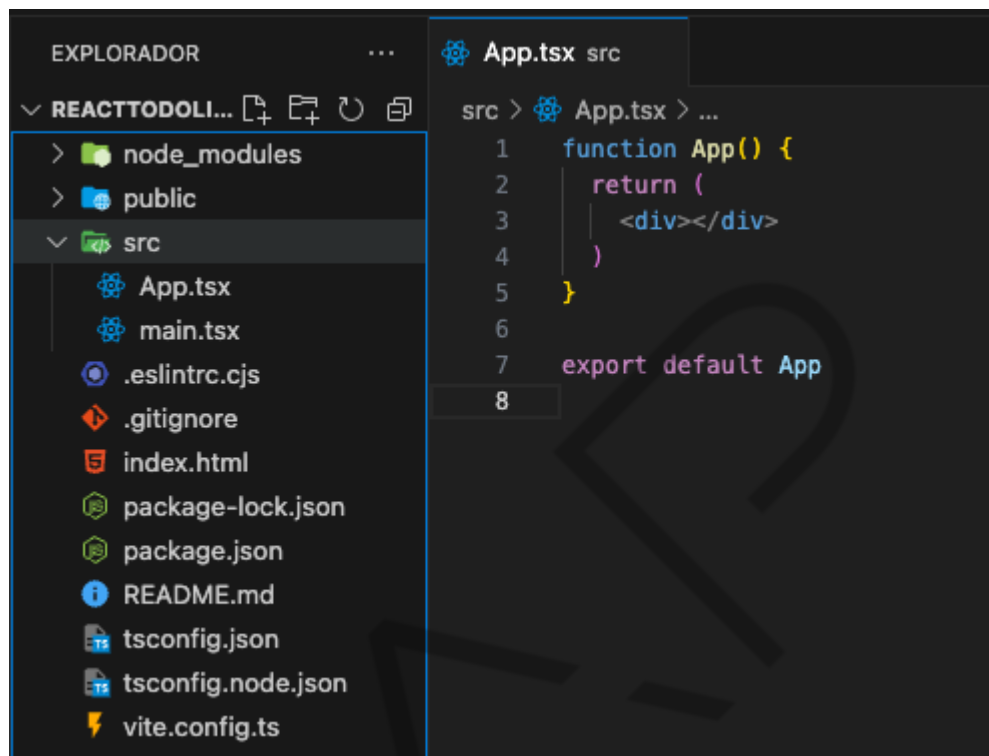


Figura 2 – VSCode
Fonte: elaborada pela autora (2024)

Vamos criar um componente filho, que receberá uma chamada do componente pai para alteração do estado inicial da propriedade **childProp**. Veja a seguir como o código ficará:

src/App.tsx

```
import { useState } from 'react';
import { ChildComponent } from './ChildComponent';

export function App() {
  const [parentState, setParentState] = useState<string>('Hello from Parent');

  const handleClick = () => {
    setParentState('Updated Parent State');
  };

  return (
    <div>
      <h1>Parent Component</h1>
      <button onClick={handleClick}>Update State</button>
      <ChildComponent childProp={parentState} />
    </div>
  );
}
```

Figura 3 – Código 2
Fonte: elaborada pela autora (2024)

src/ChildComponent.tsx

```
import { useState, useEffect } from 'react';

interface ChildProps {
  childProp: string;
}

export function ChildComponent({ childProp }: ChildProps) {
  const [childState, setChildState] = useState<string>('');

  useEffect(() => {
    setChildState(childProp);
  }, [childProp]);

  return (
    <div>
      <h2>Child Component</h2>
      <p>Child State: {childState}</p>
    </div>
  );
}
```

Figura 4 – Código 2
Fonte: elaborada pela autora (2024)

1. Inicialização:

- A aplicação é carregada.
- O estado inicial do componente pai (parentState) é definido como 'Hello from Parent'.
- O componente filho (ChildComponent) é exibido, recebendo childProp com o valor inicial de parentState.

2. Renderização inicial:

- Na primeira renderização, a tela mostra "Parent Component" com um botão chamado "Update State".
- Também exibe o "Child Component" com o texto "Child State: Hello from Parent".

3. Atualização do estado do pai:

- Quando o botão "Update State" é clicado, a função handleClick é chamada.
- setParentState('Updated Parent State') é executado, alterando o valor de parentState para 'Updated Parent State'.

4. Atualização do componente filho:

- Como o estado do componente pai foi alterado, a aplicação é re-renderizada.
- O componente filho (ChildComponent) é re-renderizado com a nova propriedade childProp={parentState}.
- Dentro do ChildComponent, o useEffect é acionado devido à mudança em childProp.
- setChildState(childProp) é executado, atualizando childState para 'Updated Parent State'.

5. Renderização atualizada:

- Agora, a tela exibe "Parent Component" com o botão "Update State" e o "Child Component" com "Child State: Updated Parent State".

O resultado fica assim:

Parent Component

Update State

Child Component

Child State: Hello from Parent

Figura 5 – Hello from Parent
Fonte: elaborada pela autora (2024)

Parent Component

Update State

Child Component

Child State: Updated Parent State

Figura 6 – Update Parent State
Fonte: elaborada pela autora (2024)

Ambiente de desenvolvimento

- [Visual Studio Code](#);
- [Node](#), de preferência a versão LTS (obrigatório para evitar problemas com pacotes);
- [Typescript](#) (instalar em ambiente global);
- [Vite](#), uma ferramenta de desenvolvimento para a construção de aplicativos web modernos. No contexto do ecossistema ReactJS, o Vite desempenha um papel fundamental, oferecendo vantagens significativas durante o desenvolvimento e facilitando a criação de aplicações mais eficientes. Consulte no site alguns dos propósitos e benefícios do Vite no ecossistema ReactJS.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, exploramos o processo de criação de componentes em React e os critérios para decidir quando criá-los. Abordamos a implementação de um projeto usando o bundler Vite, executado através de comandos no terminal. Discutimos como um componente pai pode passar estados para um componente filho e como esses estados são gerenciados e atualizados em React. Essa explicação enfatizou a importância da comunicação entre componentes pai e filho e o impacto das mudanças de estado nessa dinâmica.

Concluimos com uma visão geral sobre a transferência e atualização de propriedades em componentes em uma aplicação React, destacando o processo de mudança de estado e a interação entre componentes.

REFERÊNCIAS

ADRIANO, T. **Typescript**. São Paulo: Casa do Código, 2021.

BABEL. **Babel**. [s.d.]. Disponível em: <https://babeljs.io/>. Acesso em: 28 jun. 2024.

CARVALHO, T. **React: o que é e como funciona essa ferramenta?** Disponível em: <https://medium.com/reactbrasil/react-o-que-%C3%A9-e-como-funciona-essa-ferramenta-319922a8371c>. Acesso em: 28 jun. 2024.

REACT. **Documentação React**. [s.d.]. Disponível em: <https://react.dev/>. Acesso em: 28 jun. 2024.

PALAVRAS-CHAVE

Palavras-chave: ReactJs. Front-end. Biblioteca.

EMSE

POS TECH